Summarizer

Francisco Gomes
13/02/2023

What is a framework in Java

- It provides readymade architecture.
- It represents a set of classes and interfaces.
- It is optional.

What is Collection Framework

What is Collection framework

The Collection framework represents a unified architecture for storing and manipulating a group of objects. It has:

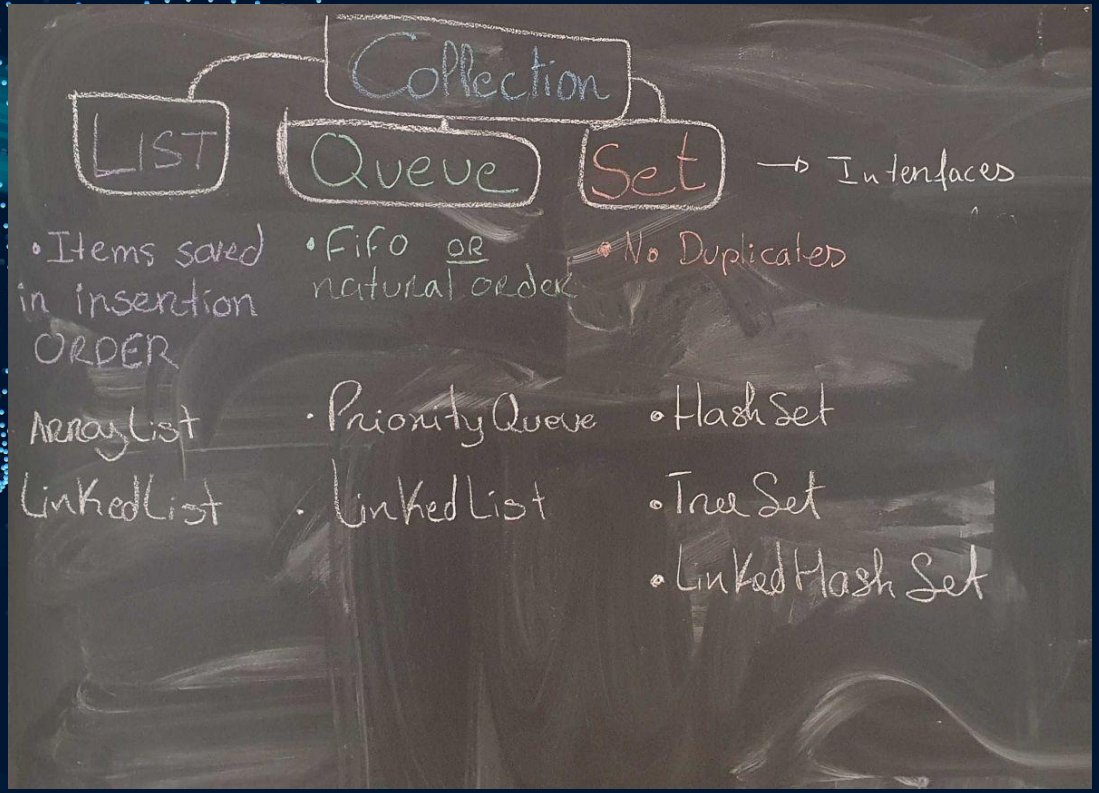
1. Interfaces and its implementations, i.e., classes
2. Algorithm

Collections

The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.

Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.

Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).


Collection
LIST
Queue
Set
→ Interfaces
• Items saved in insertion ORDER
• FiFO OR natural order
• No Duplicates
ArrayList
LinkedList
• PriorityQueue
• LinkedList
• HashSet
• TreeSet
• LinkedHashSet

Java Collections

List	Queue
Set	Map

List Interface

Holds a sequence of elements in the way they were inserted

- add() to insert objects
- get() to get them out one at a time
- iterator() to get an Iterator for the sequence.

List Methods

Add():

```
List<Integer> listOfIntegers = new ArrayList<>();
listOfIntegers.add(Integer.valueOf(100));
```

size():

```
listOfIntegers.size();
```

get():

```
listOfIntegers.get(0);
```

Queue Interface

Produces elements in an order determined by a queuing discipline, usually FIFO. Especially important in concurrent programming, because they safely transfer objects from one thread to another.

- offer() to insert objects
- peek() to get the head of the queue without removing it
- poll() to retrieve and remove the head of the queue

Queue Methods

Basic Queue Operations

The common operations that can be performed in a queue are addition, deletion & iteration. Like other collections here also we can find out the queue size & length.

The enqueue() method will add an element at the back of the queue and the dequeue() method will remove the item which is at the front of a given queue.

Set Interface

A Set can not have duplicate elements, it holds only one element of each value

- elements added to a Set must at least define equals() to establish object uniqueness.
- does not guarantee that it will maintain its elements in any particular order.

Set Methods

```
Set<Integer> setOfNumbers = new HashSet<>();
```

This Hashset is the most widely used version of a Set which gives a unique ordered list.

```
Set<String> setOfNames = new LinkedHashSet<>();
```

The only difference in LinkedHashSet with HasSet is that it orders the elements based on insertion order.

```
Set<Integer>setOfNumbers = new TreeSet<>();
```

In a TreeSet the ordering takes place based on the values of the inserted element. It follows a natural ordering by default.

Map Interface

The map is a convenient collection construct that you can use to associate one object (key) with another object (value).

Key Map must be unique and can be used later to retrieve values.

Implementations of the Map:
HashMap,
TreeMap,
LinkedHashMap

HashMap Java is the common Map type used by programmers.

Class Implementation

ToDoList

```
    private PriorityQueue<Task> priorityQueue = new PriorityQueue();



    public void add(Importance importance, Priority priority,String string){
        Task task = new Task(importance, priority, string);
        priorityQueue.add(task);
    }
    public void remove(){
        System.out.println("The task : " + priorityQueue.peek().task + ", the importance is " +
priorityQueue.peek().importance + " and the priority is " + priorityQueue.peek().priority);

        priorityQueue.remove();
    }
```

```
public enum Importance {

    HIGH(1),
    MEDIUM(2),
    LOW(3);


    int importanceLevel;

    Importance(int importanceLevel) {
        this.importanceLevel = importanceLevel;
    }


}
```

```
public enum Priority {

    TOP(1),
    MID(2),
    BOTTOM(3);

    int priorityLevel;

    Priority(int priorityLevel) {
        this.priorityLevel = priorityLevel;
    }


}
```

Task

```
public int compareTo(Task o) {
if(this.importance.importanceLevel-o.importance.importanceLevel<=1) {
    return 1;
}else if(this.importance.importanceLevel-o.importance.importanceLevel>=-1){
    return -1;
}else if (this.importance.importanceLevel-o.importance.importanceLevel==0){
    if(this.priority.priorityLevel-o.priority.priorityLevel>=-1) {
        return -1;
    }else if (this.priority.priorityLevel-o.priority.priorityLevel<=1) {
        return 1;
    }else if (this.priority.priorityLevel-o.priority.priorityLevel==0){
        return 0;
    }
```

```java
public class Main {
    public static void main(String[] args) {


        TodoList toDoList = new TodoList();

       // ToDoList todoList = new ToDoList();
       // toDoList.add(Importance.MEDIUM, Priority.A,"Clean my house");
        toDoList.add(Importance.MEDIUM, Priority.TOP,"Clean my house");
        toDoList.add(Importance.LOW, Priority.TOP, "do it");
        toDoList.add(Importance.HIGH, Priority.TOP, "almighty");
        toDoList.add(Importance.LOW, Priority.MID, "JA nem sei");
        toDoList.add(Importance.MEDIUM, Priority.MID, "tou perdido");
        toDoList.add(Importance.HIGH, Priority.MID, "perdidissimo");

       toDoList.remove();
       toDoList.remove();
       toDoList.remove();
```

Run

```
/Library/Java/JavaVirtualMachines/openjdk-8.jdk/Contents/Home/bin/java ...
The task : almighty, the importance is HIGH and the priority is TOP
The task : perdidissimo, the importance is HIGH and the priority is MID
The task : Clean my house, the importance is MEDIUM and the priority is TOP
The task : tou perdido, the importance is MEDIUM and the priority is MID
The task : do it, the importance is LOW and the priority is TOP
The task : JA nem sei, the importance is LOW and the priority is MID

Process finished with exit code 0
```

Thank you for your time!